

EDA

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import numpy as np
import pandas as pd

# get the features from the file features.txt
features = list()
with open('/content/drive/MyDrive/UCI-HAR Dataset/features.txt') as f:
    features = [line.split()[1] for line in f.readlines()]
print('No of Features: {}'.format(len(features)))
```

No of Features: 561

Getting the train data

```
# get the data from txt files to pandas dataframe
X_train = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/train/X_train.txt', sep='\s+', header=None)
X_train.columns = [features]
# add subject column to the dataframe
X_train['subject'] = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/train/subject_train.txt', header=None).

y_train = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/train/y_train.txt', names=['Activity'], header=None)
y_train_labels = y_train.map({1: 'WALKING', 2: 'WALKING_UPSTAIRS', 3: 'WALKING_DOWNSTAIRS', \
                               4: 'SITTING', 5: 'STANDING', 6: 'LAYING'})

# put all columns in a single dataframe
train = X_train
train['Activity'] = y_train
train['ActivityName'] = y_train_labels
train.sample(2)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\s'
<>:2: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-2644202008.py:2: SyntaxWarning: invalid escape sequence '\s'
X_train = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/train/X_train.txt', sep='\s+', header=None)
```

	tBodyAcc- mean()-X	tBodyAcc- mean()-Y	tBodyAcc- mean()-Z	tBodyAcc- std()-X	tBodyAcc- std()-Y	tBodyAcc- std()-Z	tBodyAcc- mad()-X	tBodyAcc- mad()-Y	tBodyAcc- mad()-Z	tBodyAcc- max()-X
629	0.234334	0.002147	-0.065574	-0.328373	-0.044268	-0.377200	-0.363918	-0.063430	-0.376386	-0.108392
6530	0.286494	-0.012064	-0.111434	-0.995578	-0.984874	-0.989941	-0.996149	-0.985007	-0.988538	-0.937069

2 rows x 564 columns

```
train.shape
```

(7352, 564)

Getting the test data

```
# get the data from txt files to pandas dataframe
X_test = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/test/X_test.txt', sep='\s+', header=None)
X_test.columns = [features]
# add subject column to the dataframe
X_test['subject'] = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/test/subject_test.txt', header=None).ilo

# get y labels from the txt file
y_test = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/test/y_test.txt', names=['Activity'], header=None)
y_test_labels = y_test.map({1: 'WALKING', 2: 'WALKING_UPSTAIRS', 3: 'WALKING_DOWNSTAIRS', \
                              4: 'SITTING', 5: 'STANDING', 6: 'LAYING'})

# put all columns in a single dataframe
test = X_test
test['Activity'] = y_test
```

```
test['ActivityName'] = y_test_labels
test.sample(2)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\s'
<>:2: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-4249596252.py:2: SyntaxWarning: invalid escape sequence '\s'
  X_test = pd.read_csv('/content/drive/MyDrive/UCI-HAR Dataset/test/X_test.txt', sep='\s+', header=None)

   tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-  tBodyAcc-
   mean()-X  mean()-Y  mean()-Z  std()-X  std()-Y  std()-Z  mad()-X  mad()-Y  mad()-Z  max()-X
2809    0.291346   -0.060765   -0.110545   -0.991207   -0.940278   -0.974095   -0.992373   -0.938120   -0.975246   -0.931358
553     0.268522   -0.042940   -0.072588   -0.407478   -0.101146   -0.646757   -0.421226   -0.097418   -0.625703   -0.208599

2 rows x 564 columns
```

```
test.shape
```

```
(2947, 564)
```

✓ Data Cleaning

✓ 0. Convert Walkings to Moving

```
# Convert ActivityName "WALKING", "WALKING_UPSTAIRS", "WALKING_DOWNSTAIRS" to "MOVING"
train["ActivityName"] = train["ActivityName"].replace(
    ["WALKING", "WALKING_UPSTAIRS", "WALKING_DOWNSTAIRS"],
    "MOVING"
)
train["ActivityName"].value_counts()
```

```
/tmp/ipython-input-3497828853.py:2: PerformanceWarning: indexing past lexsort depth may impact performance.
  train["ActivityName"] = train["ActivityName"].replace(
```

	count
(ActivityName,)	
MOVING	3285
LAYING	1407
STANDING	1374
SITTING	1286

```
dtype: int64
```

```
# Convert ActivityName "WALKING", "WALKING_UPSTAIRS", "WALKING_DOWNSTAIRS" to "MOVING"
test["ActivityName"] = test["ActivityName"].replace(
    ["WALKING", "WALKING_UPSTAIRS", "WALKING_DOWNSTAIRS"],
    "MOVING"
)
test["ActivityName"].value_counts()
```

```
/tmp/ipython-input-1274554907.py:2: PerformanceWarning: indexing past lexsort depth may impact performance.
  test["ActivityName"] = test["ActivityName"].replace(
```

	count
(ActivityName,)	
MOVING	1387
LAYING	537
STANDING	532
SITTING	491

```
dtype: int64
```

✓ JUST TRAINING

```
import numpy as np
import pandas as pd
import itertools
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
```

```

from sklearn.metrics import confusion_matrix
from datetime import datetime

from sklearn import linear_model
from sklearn import metrics

from sklearn.model_selection import GridSearchCV
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

X_train = train.drop(['subject', 'Activity', 'ActivityName'], axis=1)
y_train = train.ActivityName

X_test = test.drop(['subject', 'Activity', 'ActivityName'], axis=1)
y_test = test.ActivityName

print('X_train and y_train : ({},{})'.format(X_train.shape, y_train.shape))
print('X_test and y_test : ({},{})'.format(X_test.shape, y_test.shape))

labels = ['LAYING', 'SITTING', 'STANDING', 'MOVING']
labels

plt.rcParams["font.family"] = 'DejaVu Sans'

def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=90)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')

def perform_model(model, X_train, y_train, X_test, y_test, class_labels, cm_normalize=True, \
                  print_cm=True, cm_cmap=plt.cm.Greens):

    # to store results at various phases
    results = dict()

    # time at which model starts training
    train_start_time = datetime.now()
    print('training the model..')
    model.fit(X_train, y_train)
    print('Done....!\n')
    train_end_time = datetime.now()
    results['training_time'] = train_end_time - train_start_time
    print('==> training time:- {}\n'.format(results['training_time']))

    # predict test data
    print('Predicting test data')
    test_start_time = datetime.now()
    y_pred = model.predict(X_test)
    test_end_time = datetime.now()
    print('Done....!\n')
    results['testing_time'] = test_end_time - test_start_time
    print('==> testing time:- {}\n'.format(results['testing_time']))
    results['predicted'] = y_pred

    # calculate overall accuracy of the model
    accuracy = metrics.accuracy_score(y_true=y_test, y_pred=y_pred)
    # store accuracy in results

```

```

results['accuracy'] = accuracy
print('==> Accuracy:- {}'.format(accuracy))

# confusion matrix
cm = metrics.confusion_matrix(y_test, y_pred)
results['confusion_matrix'] = cm
if print_cm:
    print('\n *****Confusion Matrix*****')
    print('\n {}'.format(cm))

# plot confusion matrix
plt.figure(figsize=(6,6))
plt.grid(visible=False)
plot_confusion_matrix(cm, classes=class_labels, normalize=True, title='Normalized confusion matrix', cmap = cm)
plt.show()

# get classification report
print('*****| Classification Report |*****')
classification_report = metrics.classification_report(y_test, y_pred)

# store report in results
results['classification_report'] = classification_report
print(classification_report)

# add the trained model to the results
results['model'] = model

return results

def print_grid_search_attributes(model):
    # Estimator that gave highest score among all the estimators formed in GridSearch
    print('\n\n==> Best Estimator:')
    print('\t{}\n'.format(model.best_estimator_))

    # parameters that gave best results while performing grid search
    print('\n==> Best parameters:')
    print('\tParameters of best estimator : {}'.format(model.best_params_))

    # number of cross validation splits
    print('\n==> No. of CrossValidation sets:')
    print('\tTotal numbre of cross validation sets: {}'.format(model.n_splits_))

    # Average cross validated score of the best estimator, from the Grid Search
    print('\n==> Best Score:')
    print('\tAverage Cross Validate scores of best estimator : {}'.format(model.best_score_))

parameters = {'C':[0.125, 0.5, 1, 2, 8, 16]}
lr_svc = LinearSVC(tol=0.00005)
lr_svc_grid = GridSearchCV(lr_svc, param_grid=parameters, n_jobs=-1, verbose=1)
lr_svc_grid_results = perform_model(lr_svc_grid, X_train, y_train, X_test, y_test, class_labels=labels)

# observe the attributes of the model
print_grid_search_attributes(lr_svc_grid_results['model'])

```

```
X_train and y_train : ((7352, 561),(7352, 1))
X_test and y_test : ((2947, 561),(2947, 1))
training the model..
Fitting 5 folds for each of 6 candidates, totalling 30 fits
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:1408: DataConversionWarning: A column-vector
y = column_or_1d(y, warn=True)
Done....!
```

```
==> training time:- 0:00:37.874847
```

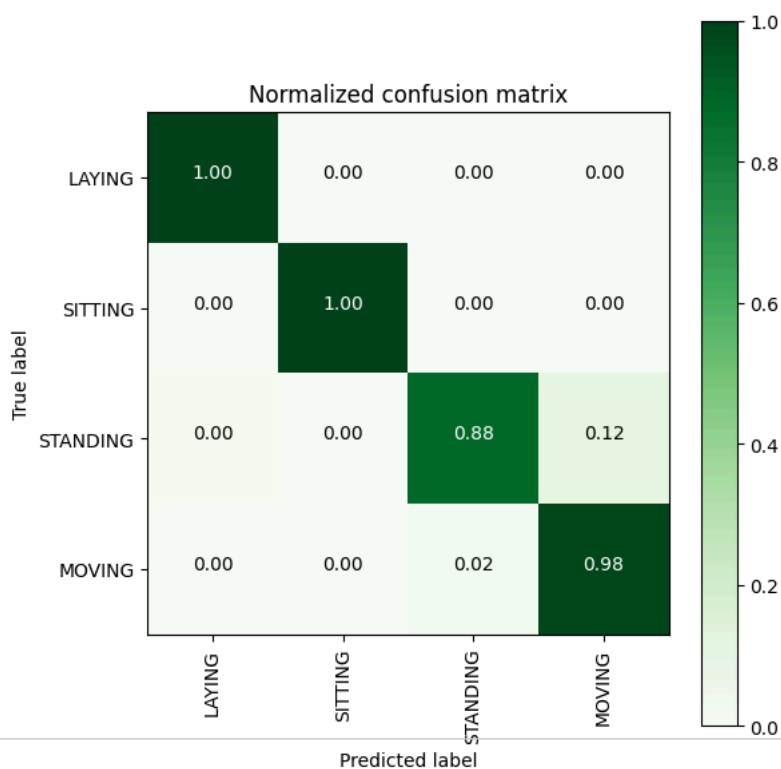
```
Predicting test data
Done....!
```

```
==> testing time:- 0:00:00.009701
```

```
==> Accuracy:- 0.9759077027485579
```

```
*****Confusion Matrix*****
```

```
[[ 537  0  0  0]
 [  0 1387  0  0]
 [  2  1 430 58]
 [  0  1  9 522]]
```



1. Check for Duplicates

```
print('No of duplicates in train: {}'.format(sum(train.duplicated())))
print('No of duplicates in test : {}'.format(sum(test.duplicated())))
```

```
/tmp/ipython-input-408555551.py:1: PerformanceWarning:
```

```
indexing past lexsort depth may impact performance
```

```
macro avg 0.97 0.96 0.97 2947
```

```
weighted avg 0.98 0.98 0.98 2947
```

```
/tmp/ipython-input-408555551.py:2: PerformanceWarning:
```

```
indexing past lexsort depth may impact performance.
```

```
==> Best Estimator:
```

```
No of duplicates in train=0
```

```
No of duplicates in test=0
```

2. Checking for NaN/null values

```
Parameters of best estimator : {'C': 0.5}
```

```
print('We have {} NaN/Null values in train'.format(train.isnull().values.sum()))
```

```
print('We have {} NaN/Null values in test'.format(test.isnull().values.sum()))
```

```
==> Best Estimator:
```

```
We have 0 NaN/Null values in train
```

```
We have 0 NaN/Null values in test
```

3. Save this dataframe in a csv files

```
import os

output_dir = 'UCI_HAR_Dataset/csv_files'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

train.to_csv(os.path.join(output_dir, 'train.csv'), index=False)
test.to_csv(os.path.join(output_dir, 'test.csv'), index=False)
```

Exploratory Data Analysis

Obtain the train and test data

```
train = pd.read_csv('UCI_HAR_Dataset/csv_files/train.csv')
test = pd.read_csv('UCI_HAR_Dataset/csv_files/test.csv')
print(train.shape, test.shape)
```

```
(7352, 564) (2947, 564)
```

All the feature name with lots of unnecessary singhs: Changing feature names

```
columns = train.columns

# Removing '()' from column names
# Using regex=True for patterns like '()', '-', ',' to ensure proper replacement
columns = columns.str.replace(r'()', '', regex=True)
columns = columns.str.replace(r'[-]', '', regex=True)
columns = columns.str.replace(r'[.,]', '', regex=True)

train.columns = columns
test.columns = columns

test.columns

Index(['tBodyAccmeanX', 'tBodyAccmeanY', 'tBodyAccmeanZ', 'tBodyAccstdX',
       'tBodyAccstdY', 'tBodyAccstdZ', 'tBodyAccmadX', 'tBodyAccmadY',
       'tBodyAccmadZ', 'tBodyAccmaxX',
       ...,
       'angletBodyAccMeangravity', 'angletBodyAccJerkMeangravityMean',
       'angletBodyGyroMeangravityMean', 'angletBodyGyroJerkMeangravityMean',
       'angleXgravityMean', 'angleYgravityMean', 'angleZgravityMean',
       'subject', 'Activity', 'ActivityName'],
      dtype='object', length=564)
```

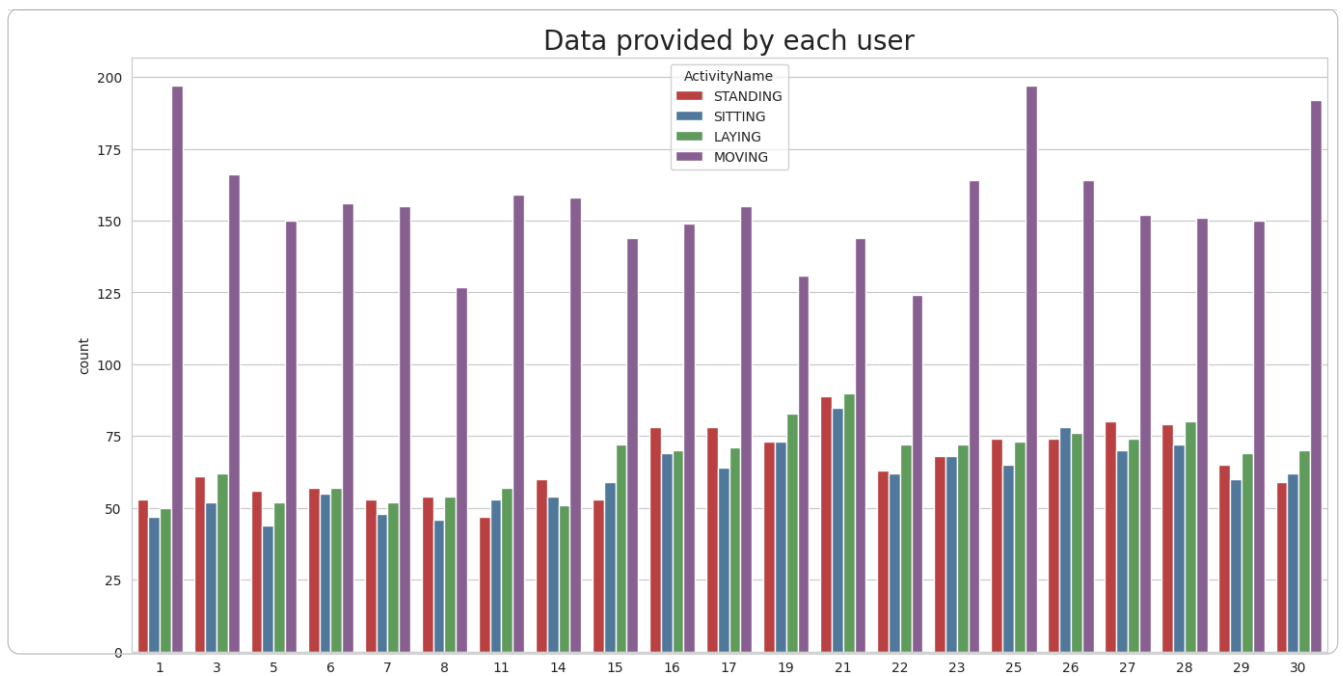
1. investigate participants activity durations.

Since the dataset has been created in a scientific environment nearly equal preconditions for the participants can be assumed. Let us investigate their activity durations.

```
import matplotlib as mlt
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
plt.rcParams['font.family'] = 'Dejavu Sans'

plt.figure(figsize=(16,8))
plt.title('Data provided by each user', fontsize=20)
sns.countplot(x='subject',hue='ActivityName', data = train)
plt.show()
```



Nearly all participants have more data for walking upstairs than downstairs. Assuming an equal number of up- and down-walks the participants need longer walking upstairs.

How Are The ActivityName Distributed?

```
import plotly
import plotly.graph_objects as go
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot

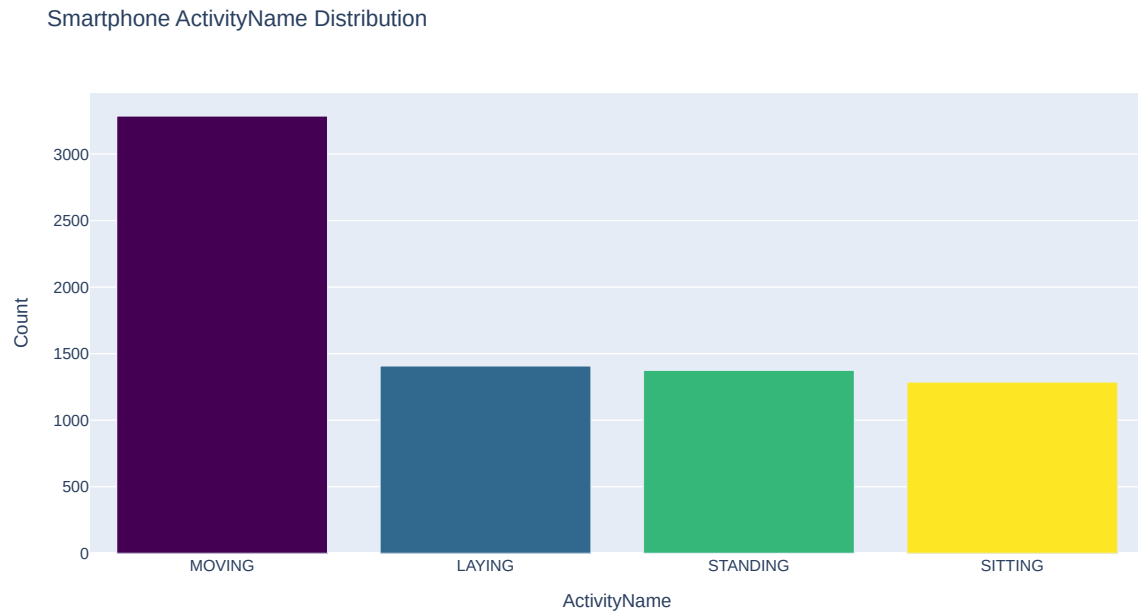
# Plotting data
label_counts = train['ActivityName'].value_counts()

# Get colors
n = label_counts.shape[0]
colormap = plt.get_cmap('viridis')
colors = [mplt.colors.to_hex(colormap(col)) for col in np.arange(0, 1.01, 1/(n-1))]

# Create plot
data = go.Bar(x = label_counts.index,
              y = label_counts,
              marker = dict(color = colors))

layout = go.Layout(title = 'Smartphone ActivityName Distribution',
                  xaxis = dict(title = 'ActivityName'),
                  yaxis = dict(title = 'Count'))

fig = go.Figure(data=[data], layout=layout)
fig.show()
#iplot(fig)
```



- Although there are fluctuations in the label counts, the labels are quite equally distributed.
- Assuming the participants had to walk the same number of stairs upwards as well as downwards and knowing the smartphones had a constant sampling rate, there should be the same amount of datapoints for walking upstairs and downstairs.
- Disregarding the possibility of flawed data, the participants seem to walk roughly 10% faster downwards.

2. Featuring Engineering from Domain Knowledge

Static and Dynamic Activities

- In static activities (sit, stand, lie down) motion information will not be very useful.
- In the dynamic activities (Walking, WalkingUpstairs, WalkingDownstairs) motion info will be significant.

3. Stationary and Moving activities are completely different

```
sns.set_palette("Set1", desat=0.80)
facetgrid = sns.FacetGrid(train, hue='ActivityName', height=5, aspect=2)
facetgrid.map(sns.distplot, 'tBodyAccMagmean', hist=False)\
    .add_legend()
plt.annotate("Stationary Activities", xy=(-0.960,12), xytext=(-0.5, 15), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))

plt.annotate("Moving Activities", xy=(0,3), xytext=(0.2, 9), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/seaborn/axisgrid.py:854: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``kdeplot`` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
/usr/local/lib/python3.12/dist-packages/seaborn/axisgrid.py:854: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``kdeplot`` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
/usr/local/lib/python3.12/dist-packages/seaborn/axisgrid.py:854: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``kdeplot`` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
/usr/local/lib/python3.12/dist-packages/seaborn/axisgrid.py:854: UserWarning:
```

``distplot`` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``kdeplot`` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

16

Stationary Activities

As expected from most real world data, when participants are moving the data is normally distributed with some long tail.

```
# for plotting purposes taking datapoints of each activity to a different dataframe
df1 = train[train['Activity']==1]
df2 = train[train['Activity']==2]
df3 = train[train['Activity']==3]
df4 = train[train['Activity']==4]
df5 = train[train['Activity']==5]
df6 = train[train['Activity']==6]

plt.figure(figsize=(14,7))
plt.subplot(2,2,1)
plt.title('Stationary Activities(Zoomed in)')
sns.distplot(df4['tBodyAccMagmean'],color = 'r',hist = False, label = 'Sitting')
sns.distplot(df5['tBodyAccMagmean'],color = 'm',hist = False,label = 'Standing')
sns.distplot(df6['tBodyAccMagmean'],color = 'c',hist = False, label = 'Laying')
plt.axis([-1.08, -0.2, 0.1, 20])
plt.legend(loc='center')

plt.subplot(2,2,2)
plt.title('Moving Activities')
sns.distplot(df1['tBodyAccMagmean'],color = 'red',hist = False, label = 'Walking')
sns.distplot(df2['tBodyAccMagmean'],color = 'blue',hist = False,label = 'Walking Up')
sns.distplot(df3['tBodyAccMagmean'],color = 'green',hist = False, label = 'Walking down')
plt.legend(loc='center right')
```

```
plt.tight_layout()
plt.show()
```

/tmp/ipython-input-1112189619.py:12: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

/tmp/ipython-input-1112189619.py:13: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

/tmp/ipython-input-1112189619.py:14: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

/tmp/ipython-input-1112189619.py:20: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

/tmp/ipython-input-1112189619.py:21: UserWarning:

We can see clearly here, the distribution of Stationary Activities and Moving Activities.

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

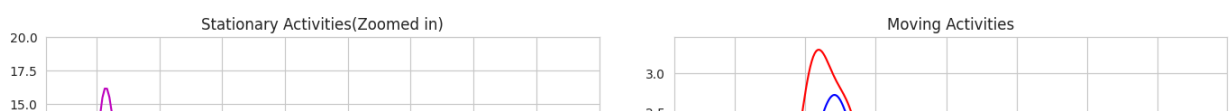
4. Magnitude of an acceleration can separate it well

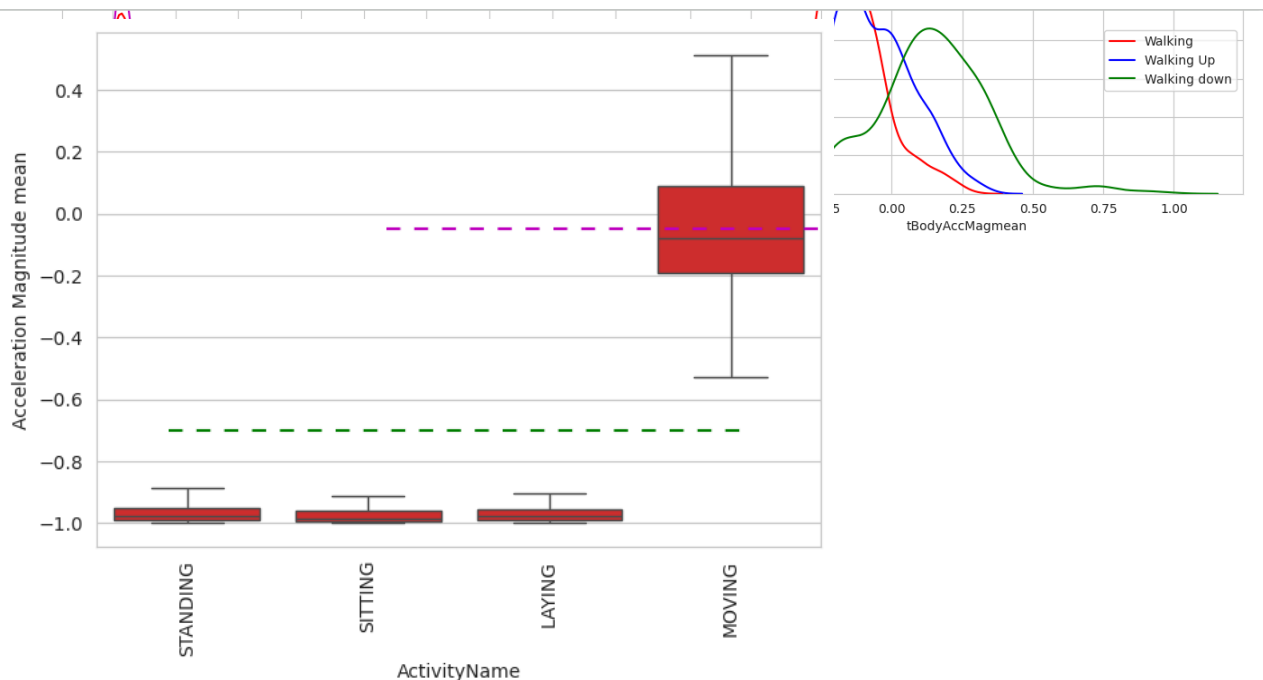
```
plt.figure(figsize=(7,5))
sns.boxplot(x='ActivityName', y='tBodyAccMagmean', data=train, showfliers=False, saturation=1)
plt.ylabel('Acceleration Magnitude mean')
plt.axhline(y=-0.7, xmin=0.1, xmax=0.9, dashes=(5,5), c='g')
plt.axhline(y=-0.05, xmin=0.4, dashes=(5,5), c='m')
plt.xticks(rotation=90)
plt.show()
```

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

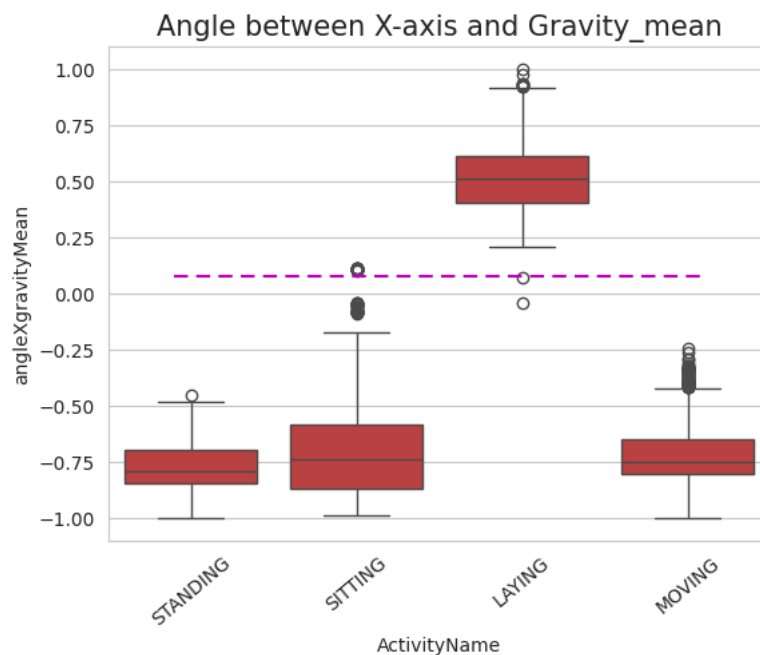




- If tAccMean is < -0.8 then the Activities are either Standing or Sitting or Laying.
- If tAccMean is > -0.6 then the Activities are either Walking or WalkingDownstairs or WalkingUpstairs.
- If tAccMean > 0.0 then the Activity is WalkingDownstairs.
- We can classify 75% the Activity labels with some errors.

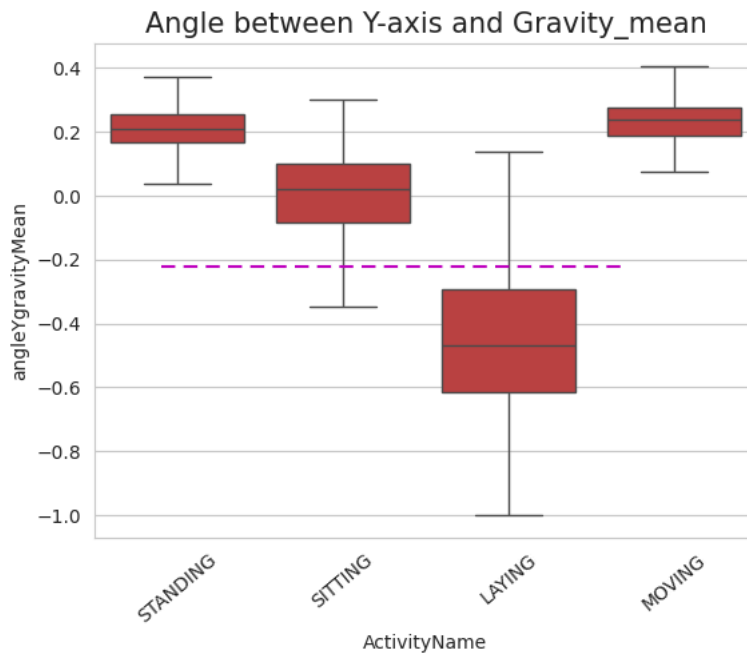
5. Position of GravityAccelerationComponents also matters

```
sns.boxplot(x='ActivityName', y='angleXgravityMean', data=train)
plt.axhline(y=0.08, xmin=0.1, xmax=0.9, c='m', dashes=(5,3))
plt.title('Angle between X-axis and Gravity_mean', fontsize=15)
plt.xticks(rotation = 40)
plt.show()
```



- If angleXgravityMean > 0 then Activity is Laying.
- We can classify all datapoints belonging to Laying activity with just a single if else statement.

```
sns.boxplot(x='ActivityName', y='angleYgravityMean', data = train, showfliers=False)
plt.title('Angle between Y-axis and Gravity_mean', fontsize=15)
plt.xticks(rotation = 40)
plt.axhline(y=-0.22, xmin=0.1, xmax=0.8, dashes=(5,3), c='m')
plt.show()
```



▼ Apply t-sne on the data

Are The Activities Separable?

The dataset is geared towards classifying the activity of the participant. Let us investigate the separability of the classes.

```
import numpy as np
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# performs t-sne with different perplexity values and their repective plots..

def perform_tsne(X_data, y_data, perplexities, n_iter=1000, img_name_prefix='t-sne'):

    for index,perplexity in enumerate(perplexities):
        # perform t-sne
        print('\nperforming tsne with perplexity {} and with {} iterations at max'.format(perplexity, n_iter))
        X_reduced = TSNE(verbose=2, perplexity=perplexity).fit_transform(X_data)
        print('Done..')

        # prepare the data for seaborn
        print('Creating plot for this t-sne visualization..')
        df = pd.DataFrame({'x':X_reduced[:,0], 'y':X_reduced[:,1] , 'label':y_data})

        # draw the plot in appropriate place in the grid
        sns.lmplot(data=df, x='x', y='y', hue='label', fit_reg=False, height=8,\
                    palette="Set1",markers=['^','v','s','o', '1','2'])
        plt.title("perplexity : {} and max_iter : {}".format(perplexity, n_iter))
        img_name = img_name_prefix + '_perp_{}_iter_{}.png'.format(perplexity, n_iter)
        print('saving this plot as image in present working directory...')
        plt.savefig(img_name)
        plt.show()
        print('Done')
```

```
X_pre_tsne = train.drop(['subject', 'Activity','ActivityName'], axis=1)
y_pre_tsne = train['ActivityName']
perform_tsne(X_data = X_pre_tsne,y_data=y_pre_tsne, perplexities =[2,5,10,20,50])
```

```

performing tsne with perplexity 2 and with 1000 iterations at max
[t-SNE] Computing 7 nearest neighbors...
[t-SNE] Indexed 7352 samples in 0.016s...
[t-SNE] Computed neighbors for 7352 samples in 1.423s...
[t-SNE] Computed conditional probabilities for sample 1000 / 7352
[t-SNE] Computed conditional probabilities for sample 2000 / 7352
[t-SNE] Computed conditional probabilities for sample 3000 / 7352
[t-SNE] Computed conditional probabilities for sample 4000 / 7352
[t-SNE] Computed conditional probabilities for sample 5000 / 7352
[t-SNE] Computed conditional probabilities for sample 6000 / 7352
[t-SNE] Computed conditional probabilities for sample 7000 / 7352
[t-SNE] Computed conditional probabilities for sample 7352 / 7352
[t-SNE] Mean sigma: 0.597443
[t-SNE] Computed conditional probabilities in 0.014s
[t-SNE] Iteration 50: error = 119.3308029, gradient norm = 0.0433288 (50 iterations in 6.197s)
[t-SNE] Iteration 100: error = 105.1872559, gradient norm = 0.0247335 (50 iterations in 2.274s)
[t-SNE] Iteration 150: error = 99.8314056, gradient norm = 0.0172896 (50 iterations in 1.983s)
[t-SNE] Iteration 200: error = 96.7782974, gradient norm = 0.0134763 (50 iterations in 1.940s)
[t-SNE] Iteration 250: error = 94.6483154, gradient norm = 0.0119565 (50 iterations in 2.442s)
[t-SNE] KL divergence after 250 iterations with early exaggeration: 94.648315
[t-SNE] Iteration 300: error = 4.2690125, gradient norm = 0.0181472 (50 iterations in 2.635s)
[t-SNE] Iteration 350: error = 3.3594513, gradient norm = 0.0191954 (50 iterations in 1.863s)
[t-SNE] Iteration 400: error = 2.8625464, gradient norm = 0.0154119 (50 iterations in 1.882s)
[t-SNE] Iteration 450: error = 2.5696101, gradient norm = 0.0133376 (50 iterations in 1.916s)
[t-SNE] Iteration 500: error = 2.3693209, gradient norm = 0.0113140 (50 iterations in 1.953s)
[t-SNE] Iteration 550: error = 2.2202444, gradient norm = 0.0102599 (50 iterations in 2.092s)
[t-SNE] Iteration 600: error = 2.1037979, gradient norm = 0.0092914 (50 iterations in 3.102s)
[t-SNE] Iteration 650: error = 2.0087998, gradient norm = 0.0085030 (50 iterations in 1.956s)
[t-SNE] Iteration 700: error = 1.9294534, gradient norm = 0.0079321 (50 iterations in 1.923s)
[t-SNE] Iteration 750: error = 1.8615533, gradient norm = 0.0073962 (50 iterations in 1.919s)
[t-SNE] Iteration 800: error = 1.8023643, gradient norm = 0.0069954 (50 iterations in 1.938s)
[t-SNE] Iteration 850: error = 1.7506326, gradient norm = 0.0066360 (50 iterations in 1.932s)
[t-SNE] Iteration 900: error = 1.7044195, gradient norm = 0.0063298 (50 iterations in 2.961s)
[t-SNE] Iteration 950: error = 1.6630683, gradient norm = 0.0062102 (50 iterations in 2.207s)
[t-SNE] Iteration 1000: error = 1.6256750, gradient norm = 0.0059814 (50 iterations in 1.949s)
[t-SNE] KL divergence after 1000 iterations: 1.625675
Done..
Creating plot for this t-sne visualization..

```

```

ValueError                                Traceback (most recent call last)

```

```

/tmp/ipython-input-2076427964.py in <cell line: 0>()
      1 X_pre_tsne = train.drop(['subject', 'Activity', 'ActivityName'], axis=1)
      2 y_pre_tsne = train['ActivityName']
----> 3 perform_tsne(X_data = X_pre_tsne, y_data=y_pre_tsne, perplexities =[2,5,10,20,50])

```

1 frames

```

/usr/local/lib/python3.12/dist-packages/seaborn/regression.py in lmplot(data, x, y, hue, col, row, palette,
col_wrap, height, aspect, markers, sharey, sharex, hue_order, col_order, row_order, legend, legend_out,
x_estimator, x_bins, x_ci, scatter, fit_reg, ci, n_boot, units, seed, order, logistic, lowess, robust, logx,
x_partial, y_partial, truncate, x_jitter, y_jitter, scatter_kws, line_kws, facet_kws)
    629     markers = [markers] * n_markers
    630     if len(markers) != n_markers:
--> 631         raise ValueError("markers must be a singleton or a list of markers "

```

We can clearly see the TSNE cluster. All the Activity are clean separate except "Standing" and "Sitting".

ValueError: markers must be a singleton or a list of markers for each level of the hue variable

Train Test Split

1.0

```

X_train = train.drop(['subject', 'Activity', 'ActivityName'], axis=1)
y_train = train.ActivityName

X_test = test.drop(['subject', 'Activity', 'ActivityName'], axis=1)
y_test = test.ActivityName

print('X_train and y_train : ({},{})'.format(X_train.shape, y_train.shape))
print('X_test and y_test : ({},{})'.format(X_test.shape, y_test.shape))

```

```

X_train and y_train : ((7352, 561),(7352,))
X_test and y_test : ((2947, 561),(2947,))

```

Define Labels that are useful in plotting confusion matrix

0.6

```

labels = ['LAYING', 'SITTING', 'STANDING', 'WALKING', 'WALKING_DOWNSTAIRS', 'WALKING_UPSTAIRS']
labels

```

```

['LAYING',
 'SITTING',
 'STANDING',
 0.4]

```

```
'WALKING',
'WALKING_DOWNSTAIRS',
'WALKING_UPSTAIRS']
```

function to plot the confusion matrix

0.2

```
plt.rcParams["font.family"] = 'DejaVu Sans'

def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):

    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=90)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
```

function to run any model specified

```
from datetime import datetime
def perform_model(model, X_train, y_train, X_test, y_test, class_labels, cm_normalize=True, \
                  print_cm=True, cm_cmap=plt.cm.Greens):

    # to store results at various phases
    results = dict()

    # time at which model starts training
    train_start_time = datetime.now()
    print('training the model..')
    model.fit(X_train, y_train)
    print('Done...!\n')
    train_end_time = datetime.now()
    results['training_time'] = train_end_time - train_start_time
    print('==> training time:- {}'.format(results['training_time']))

    # predict test data
    print('Predicting test data')
    test_start_time = datetime.now()
    y_pred = model.predict(X_test)
    test_end_time = datetime.now()
    print('Done...!\n')
    results['testing_time'] = test_end_time - test_start_time
    print('==> testing time:- {}'.format(results['testing_time']))
    results['predicted'] = y_pred

    # calculate overall accuracy of the model
    accuracy = metrics.accuracy_score(y_true=y_test, y_pred=y_pred)
    # store accuracy in results
    results['accuracy'] = accuracy
    print('==> Accuracy:- {}'.format(accuracy))

    # confusion matrix
    cm = metrics.confusion_matrix(y_test, y_pred)
    results['confusion_matrix'] = cm
    if print_cm:
        print('\n *****Confusion Matrix*****')
        print('\n {}'.format(cm))
```

```

# plot confusin matrix
plt.figure(figsize=(6,6))
plt.grid(visible=False)
plot_confusion_matrix(cm, classes=class_labels, normalize=True, title='Normalized confusion matrix', cmap =
plt.show())

# get classification report
print('*****| Classification Report |*****')
classification_report = metrics.classification_report(y_test, y_pred)

# store report in results
results['classification_report'] = classification_report
print(classification_report)

# add the trained model to the results
results['model'] = model

return results

```

✓ function to print the gridsearch Parameters

```

def print_grid_search_attributes(model):
    # Estimator that gave highest score among all the estimators formed in GridSearch
    print('\n\n==> Best Estimator:')
    print('\t{}\n'.format(model.best_estimator_))

    # parameters that gave best results while performing grid search
    print('\n==> Best parameters:')
    print('\tParameters of best estimator : {}'.format(model.best_params_))

    # number of cross validation splits
    print('\n==> No. of CrossValidation sets:')
    print('\tTotal numbre of cross validation sets: {}'.format(model.n_splits_))

    # Average cross validated score of the best estimator, from the Grid Search
    print('\n==> Best Score:')
    print('\tAverage Cross Validate scores of best estimator : {}'.format(model.best_score_))

```

✓ 1. Logistic Regression with Grid Search

```

import numpy as np
import pandas as pd
import itertools
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from datetime import datetime

from sklearn import linear_model
from sklearn import metrics

from sklearn.model_selection import GridSearchCV
from sklearn.svm import LinearSVC
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

import warnings
from sklearn.exceptions import ConvergenceWarning

```

```

warnings.filterwarnings("ignore")
warnings.simplefilter(action='ignore', category=ConvergenceWarning)

# start Grid search
parameters = {'C':[0.01, 0.1, 1, 10, 20, 30], 'penalty':['l2','l1']}
log_reg = linear_model.LogisticRegression()
log_reg_grid = GridSearchCV(log_reg, param_grid=parameters, cv=3, verbose=1, n_jobs=-1)
log_reg_grid_results = perform_model(log_reg_grid, X_train, y_train, X_test, y_test, class_labels=labels)

# observe the attributes of the model
print_grid_search_attributes(log_reg_grid_results['model'])

```

```
training the model..
Fitting 3 folds for each of 12 candidates, totalling 36 fits
Done....!
```

```
==> training time:- 0:00:23.326939
```

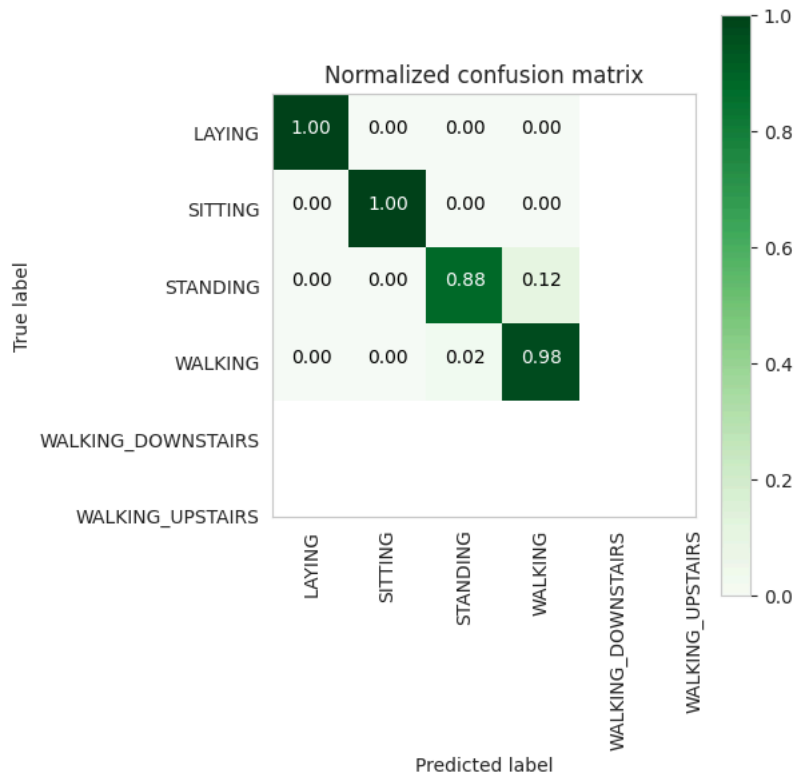
```
Predicting test data
Done....!
```

```
==> testing time:- 0:00:00.017726
```

```
==> Accuracy:- 0.9755683746182559
```

```
*****Confusion Matrix*****
```

```
[[ 537    0    0    0]
 [   0 1387    0    0]
 [   0    1  432   58]
 [   0    1   12  519]]
```



```
*****| Classification Report |*****
precision    recall  f1-score   support
```

✓ **2. Linear SVC with GridSearch**

	precision	recall	f1-score	support
LAYING	1.00	1.00	1.00	537
SITTING	1.00	1.00	1.00	1387
STANDING	0.97	0.88	0.92	491
WALKING	0.90	0.98	0.94	532

```
parameters = {'C':[0.125, 0.5, 1, 2, 8, 16]}
lr_svc = LinearSVC(tol=0.00005)
lr_svc_grid = GridSearchCV(lr_svc, param_grid=parameters, n_jobs=-1, verbose=1)
lr_svc_grid_results = perform_model(lr_svc_grid, X_train, y_train, X_test, y_test, class_labels=labels)
```

```
# observe the attributes of the model
print_grid_search_attributes(lr_svc_grid_results['model'])
LogisticRegression(C=1)
```

```
==> Best parameters:
Parameters of best estimator : {'C': 1, 'penalty': 'l2'}
```

```
==> No. of CrossValidation sets:
Total nombre of cross validation sets: 3
```

```
==> Best Score:
Average Cross Validate scores of best estimator : 0.9706205713619598
```

```
training the model..
Fitting 5 folds for each of 6 candidates, totalling 30 fits
Done....!
```

```
==> training time:- 0:00:38.549518
```

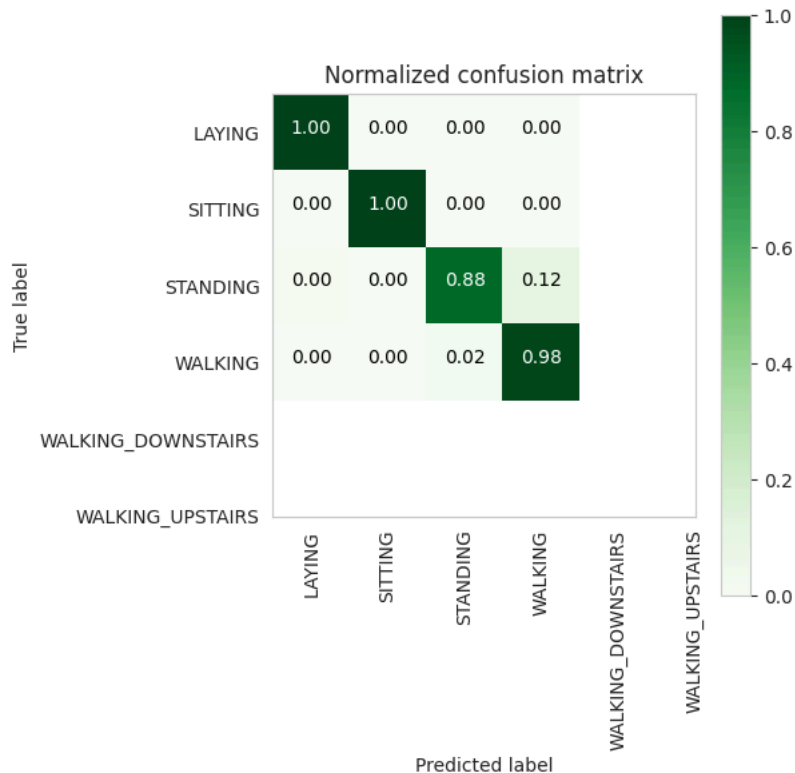
```
Predicting test data
Done....!
```

```
==> testing time:- 0:00:00.010091
```

```
==> Accuracy:- 0.9759077027485579
```

```
*****Confusion Matrix*****
```

```
[[ 537    0    0    0]
 [   0 1387    0    0]
 [    2    1 430   58]
 [    0    1    9 522]]
```



```
*****| Classification Report |*****
precision    recall  f1-score   support
```

```

LAYING      1.00      1.00      1.00      537
SITTING     1.00      1.00      1.00     1387
STANDING    0.98      0.88      0.92      491
WALKING     0.90      0.98      0.94      532
```

3. Kernel SVM with GridSearch

```
parameters = {'C':[2,8,16],\
              'gamma':[ 0.0078125, 0.125, 2]}
rbf_svm = SVC(kernel='rbf')
rbf_svm_grid = GridSearchCV(rbf_svm,param_grid=parameters, n_jobs=-1)
rbf_svm_grid_results = perform_model(rbf_svm_grid, X_train, y_train, X_test, y_test, class_labels=labels)

# observe the attributes of the model
print_grid_search_attributes(rbf_svm_grid_results['model'])
```

```
==> Best parameters:
Parameters of best estimator : {'C': 0.5}
```

```
==> No. of CrossValidation sets:
Total nombre of cross validation sets: 5
```

```
==> Best Score:
Average Cross Validate scores of best estimator : 0.975653750283254
```

```

training the model..
Done....!

==> training time:- 0:06:29.780937

Predicting test data
Done....!

==> testing time:- 0:00:01.006939

==> Accuracy:- 0.9793009840515778

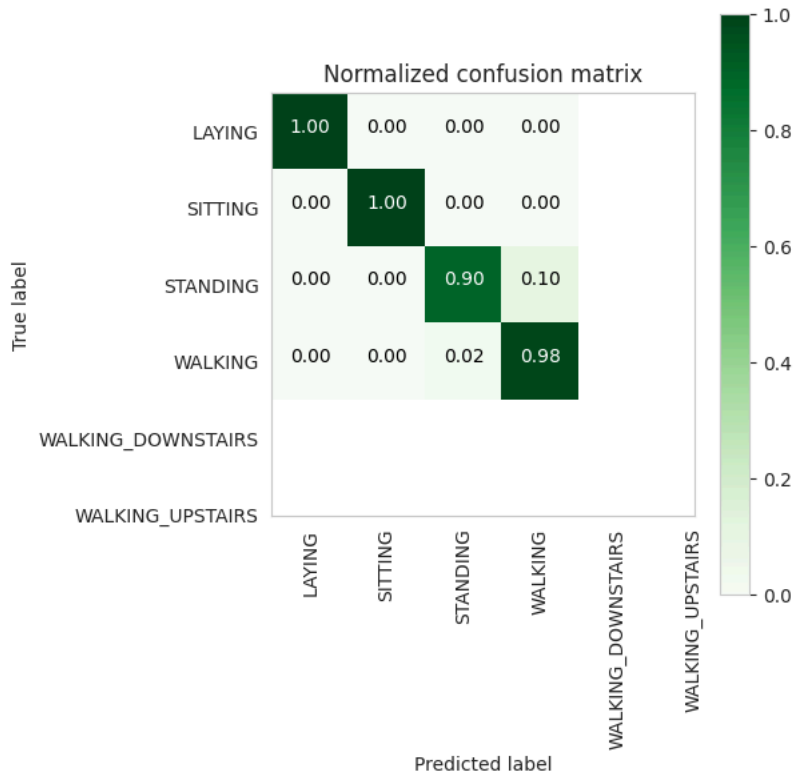
```

*****Confusion Matrix*****

```

[[ 537    0    0    0]
 [    0 1387    0    0]
 [    0    1  442  48]
 [    0    0   12 520]]

```



*****| Classification Report |*****

	precision	recall	f1-score	support
LAYING	1.00	1.00	1.00	537
SITTING	1.00	1.00	1.00	1387
STANDING	0.99	0.90	0.94	491
WALKING	0.92	0.98	0.95	532

4. Decision Trees with GridSearchCV

```

parameters = {'max_depth':np.arange(3,10,2)}
dt = DecisionTreeClassifier()
dt_grid = GridSearchCV(dt,param_grid=parameters, n_jobs=-1)
dt_grid_results = perform_model(dt_grid, X_train, y_train, X_test, y_test, class_labels=labels)

# observe the attributes of the model
print_grid_search_attributes(dt_grid_results['model'])

==> Best parameters:
Parameters of best estimator : {'C': 16, 'gamma': 0.0078125}

==> No. of CrossValidation sets:
Total number of cross validation sets: 5

==> Best Score:
Average Cross Validate scores of best estimator : 0.9755172334059388

```

```

training the model..
Done....!

==> training time:- 0:00:27.819411

Predicting test data
Done....!

==> testing time:- 0:00:00.006480

==> Accuracy:- 0.9328130302002036

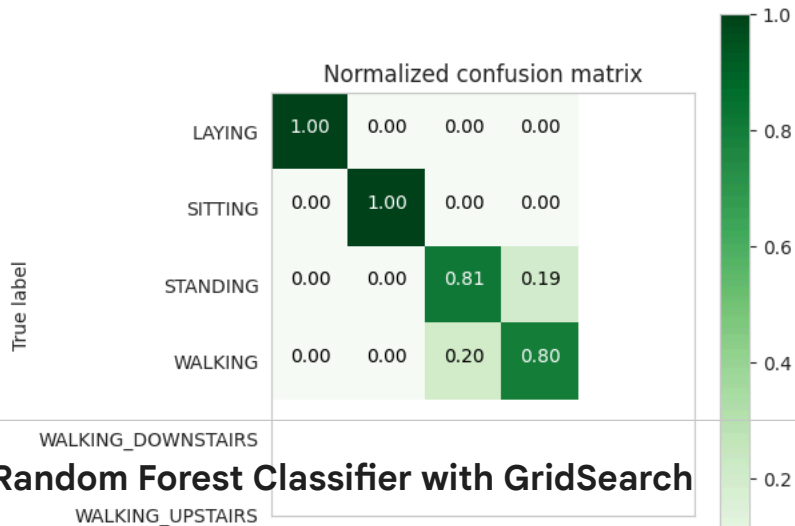
```

*****Confusion Matrix*****

```

[[ 537    0    0    0]
 [   0 1387    0    0]
 [   0    0 400   91]
 [   0    0 107  425]]

```



5. Random Forest Classifier with GridSearch

```

params = {'n_estimators': np.arange(10,201,20), 'max_depth':np.arange(3,15,2)}
rfc = RandomForestClassifier()
rfc_grid = GridSearchCV(rfc, param_grid=params, n_jobs=-1)
rfc_grid_results = perform_model(rfc_grid, X_train, y_train, X_test, y_test, class_labels=labels)

# observe the attributes of the model
print_grid_search_attributes(rfc_grid_results['model'])

```

```

training the model..
Done.....!
*****| Classification Report |*****
precision    recall  f1-score   support
-- training time: 0:25:20.136100

```